

Individual assignment cover sheet

Course Name:	Master of Information Systems (Cyber Security)
Unit Number and Name:	MIS5303 Securing Software
Lecturer Name:	Dr Omid Haass
Assignment No.:	Assessment 1 - Practical Labs (Lab 3)

Declaration:

By submitting this declaration, I, the below-named student, declare the following:

- ✓ Except where appropriately acknowledged, this assignment is my own work and has been expressed in my own words.
- ✓ Where I have made use of generative AI software, I have done so appropriately and cited it accurately as per the '**HE-AP19 Generative Artificial Intelligence (AI) Guidelines for Students**' document found in the LMS, See: [HE-AP19 Generative Artificial Intelligence \(AI\) Guidelines for Students.pdf \(apexhe.nsw.edu.au\)](https://apexhe.nsw.edu.au/HE-AP19%20Generative%20Artificial%20Intelligence%20(AI)%20Guidelines%20for%20Students.pdf).
- ✓ This assignment in its entirety, or this particular part of it, has not previously been submitted by me or any other person for assessment on this or any other course of study.
- ✓ I have read the Student Academic Misconduct Policy and Procedure found at <https://apexhe.nsw.edu.au/policiesandprocedures/>, and I understand the significance of academic integrity and the consequences of breaching it.

Use of Generative Artificial Intelligence (AI)

Please tick one of the following options:

- ✓ **Declaration 1:** I acknowledge using AI tool(s) to generate content that are included in this work in modified forms.
- ✓ **Declaration 2:** I acknowledge the use of AI tool(s) in the preparation of material that aids the process of completion of this assignment (e.g. for self-study or background reading). No materials are submitted as original work on our part.
- ✓ **Declaration 3:** I did not use any content generated by AI tools in the preparation of this assignment

If your response was **Declaration 1** or **Declaration 2**, please complete the following:

	Student's Response
AI Tool(s) Used (e.g., ChatGPT, Grammarly, DALL·E 2, Pi Assistant, Lumen5, Looka, etc.)	ChatGPT
Purpose of Use (Explain specifically what the tool was used for – e.g., idea generation, grammar correction, image generation, summarizing content, etc.)	• Concept grasping. • Error explanation. • Testing Result verification. • Formatting Doc file

List all prompts you have used	
Extent of Modification (Explain if and how you rewrote or modified the AI-generated content. E.g., rephrased, fact-checked, rearranged, quoted suitably, etc.)	
Consider the manners in which your work is different from AI-generated contents.	• Writing the explanation in my own words. • Attaching the work-screenshots. • Inserting workflow diagram. • Executions are done by me in cmd.
How have you modified the generative AI's output?	I write in my own words.

Note: All AI usage must be acknowledged and comply with the HE-AP19 Generative Artificial Intelligence (AI) Guidelines for Students. Unacknowledged use of AI tools constitutes academic misconduct.

Student ID Number:	Student Name:
240135	Ashraful Islam

Signed:

Date: 01-Aug-2026



Recoverable Signature

X Ashraful

Ashraful Islam

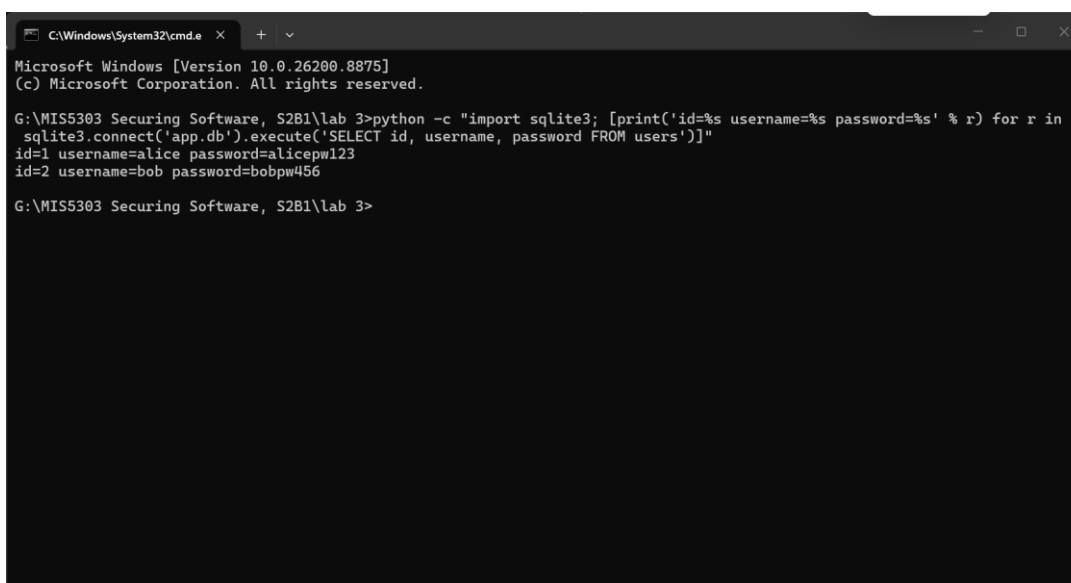
Signed by: 341b0587-a755-4806-a88b-56fe95828d8f

1. Findings — Auth, Session & Privilege Flaws

ID	Vulnerability	Where	OWASP	Severity	Why it is risky
F1	Plaintext passwords	register() / app.db	A02:2021	High	Anyone who reads the DB (backup leak, injection, insider) gets every password instantly; reuse spreads the damage
F2	Insecure session cookie + fixation	app.config / login()	A05/A07	Medium	HttpOnly off lets JavaScript steal the cookie; no session.clear() lets a planted session id be reused after login
F3	Privilege escalation / broken access control	http://127.0.0.1:5000/http://127.0.0.1:5000/make_admin, http://127.0.0.1:5000/http://127.0.0.1:5000/admin	A01:2021	High	A normal user reaches http://127.0.0.1:5000/admin and self-promotes via http://127.0.0.1:5000/make_admin?user_id=; full admin takeover

F1 — Plaintext passwords (before)

Step: I opened app.db and listed the users table. The passwords are stored exactly as typed, with no hashing. If the database is ever leaked, every account is compromised at once.



```
C:\Windows\System32\cmd.exe X + v
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

G:\MIS5303 Securing Software, S2B1\lab 3>python -c "import sqlite3; [print('id=%s username=%s password=%s' % r) for r in
sqlite3.connect('app.db').execute('SELECT id, username, password FROM users')]"
id=1 username=alice password=alicepw123
id=2 username=bob password=bobpw456

G:\MIS5303 Securing Software, S2B1\lab 3>
```

Figure 1: users table in the vulnerable app — passwords stored in plaintext (alicepw123, bobpw456).

F2 — Session cookie missing flags (before)

Step: logged in as a normal user, I opened DevTools > Application > Cookies. The session cookie has HttpOnly, Secure and SameSite all empty. The login route also never calls session.clear(), so the pre-login session id is reused (session fixation).

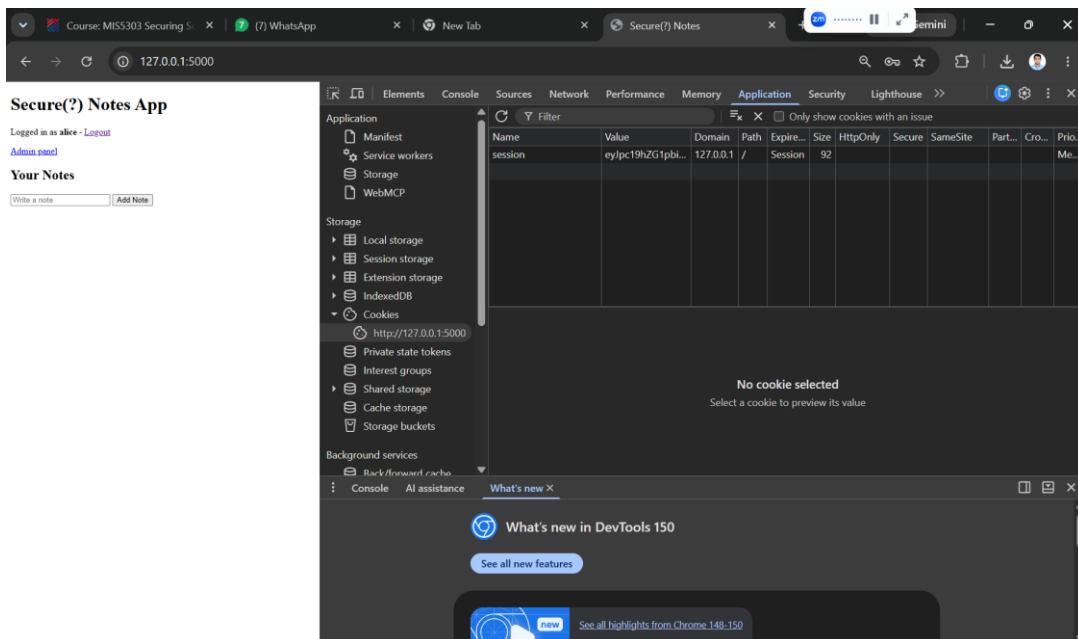


Figure 2: session cookie in the vulnerable app — HttpOnly, Secure and SameSite are all unset.

F3 — Privilege escalation (before)

Step: logged in as the normal user 'alice', I opened `http://127.0.0.1:5000/admin` — the panel loads even though alice is not an admin (broken access control). I then visited `http://127.0.0.1:5000/make_admin?user_id=1`, which promoted alice to admin. The panel now shows alice as admin=1.

```

C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

G:\MIS5303 Securing Software, S2B1\lab 3>python -c "import sqlite3; [print('id=%s username=%s password=%s' % r) for r in
sqlite3.connect('app.db').execute('SELECT id, username, password FROM users')]"
id=1 username=alice password=$2b$12$telIS0LVS.NxXA0ruF4P10Dd0/iszWSyWFLbbZXaJDU15bUR6rGAu
G:\MIS5303 Securing Software, S2B1\lab 3>

```

Figure 3: after visiting `http://127.0.0.1:5000/make_admin?user_id=1`, the normal user 'alice' has escalated herself to admin=1.

2. Remediation — Secure Code (Before / After)

2.1 Password hashing (bcrypt)

Before:

```
conn.execute("INSERT INTO users (username,password) VALUES (?,?)", (username, password))
```

After:

```
pw_hash = bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt(rounds=12)).decode("utf-8")
conn.execute("INSERT INTO users (username,password) VALUES (?,?)", (username, pw_hash))
# login: bcrypt.checkpw(password.encode(), user["password"].encode())
```

Why: bcrypt is deliberately slow and salts every password, so identical passwords hash differently and offline brute-force is far harder than with a fast hash such as MD5/SHA-256.

2.2 Session hardening + fixation fix

```
app.config.update(SESSION_COOKIE_HTTPONLY=True, SESSION_COOKIE_SAMESITE="Lax",
                  SESSION_COOKIE_SECURE=False, PERMANENT_SESSION_LIFETIME=timedelta(minutes=15))
session.clear(); session.permanent = True # on login: defeats fixation + idle timeout
```

Limitation noted: SESSION_COOKIE_SECURE must be True in production behind HTTPS. On plain http://127.0.0.1 the browser would then refuse to send the cookie, so it is False for local testing only — a documented environment limitation, not a code fault.

2.3 Access control (fix http://127.0.0.1:5000/admin and http://127.0.0.1:5000/make_admin)

```
def admin_required(view):
    @wraps(view)
    def wrapper(*a, **k):
        if not session.get("username"): return redirect(url_for("login"))
        row = get_db().execute("SELECT is_admin FROM users WHERE username=?",
                               (session["username"],)).fetchone()
        if not row or not row["is_admin"]: abort(403)
        return view(*a, **k)
    return wrapper
@app.route("http://127.0.0.1:5000/make_admin", methods=["POST"]) # POST-only
@admin_required # admin-only
```

Two design choices worth stating: the admin flag is read from the DATABASE (authoritative), not trusted from the client-held session; and http://127.0.0.1:5000/make_admin is POST-only so it cannot be triggered by a crafted link (GET/CSRF).

Re-test — after remediation

I repeated every exploit against the fixed app. All are now blocked:

Exploit repeated	Before	After (fixed)
Read passwords in app.db	plaintext visible	bcrypt hashes (\$2b\$12\$...)
Normal user opens http://127.0.0.1:5000/admin	HTTP 200 (panel loads)	HTTP 403 (denied)
GET http://127.0.0.1:5000/make_admin?user_id=	promotes to admin	HTTP 405 (GET blocked)
Login with correct password	works	still works (hash verified)
Session cookie flags	HttpOnly/SameSite empty	HttpOnly + SameSite set

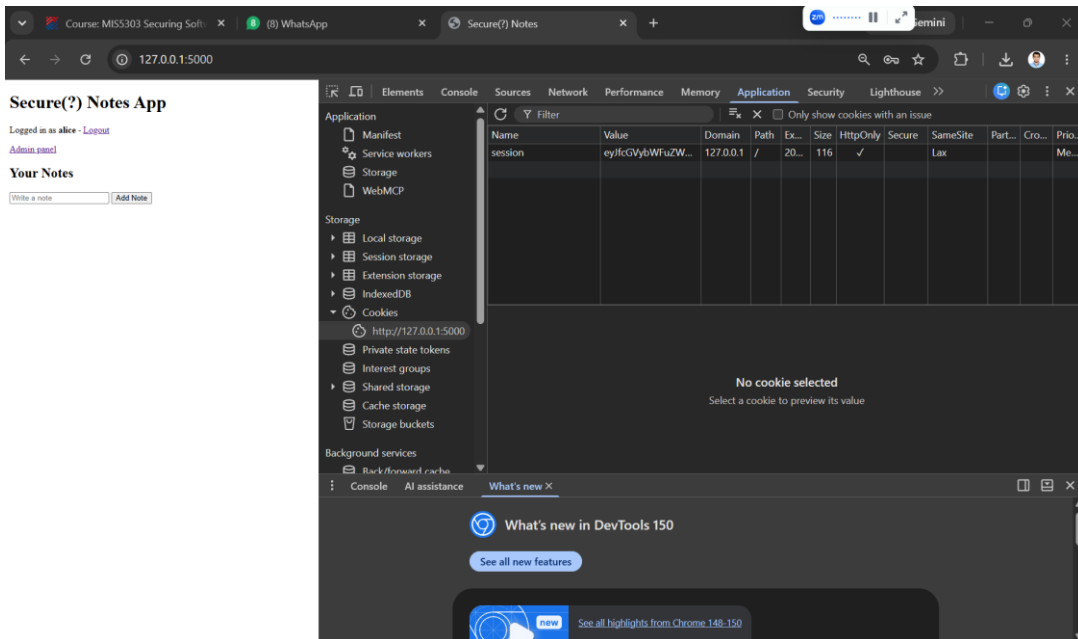


Figure 4: after the fix — the same user's password is now stored as a bcrypt hash (\$2b\$12\$...), not plaintext.

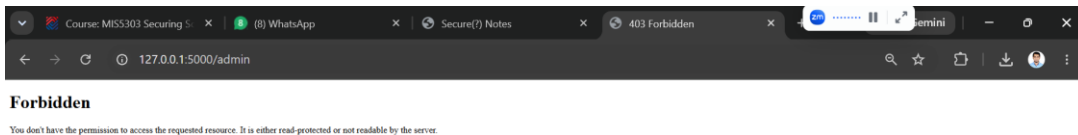


Figure 5: after the fix — the session cookie now has HttpOnly (checkmark) and SameSite = Lax set.

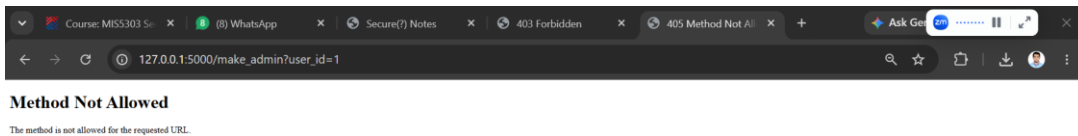


Figure 6: after the fix — the normal user 'alice' is denied the admin panel with HTTP 403 Forbidden.

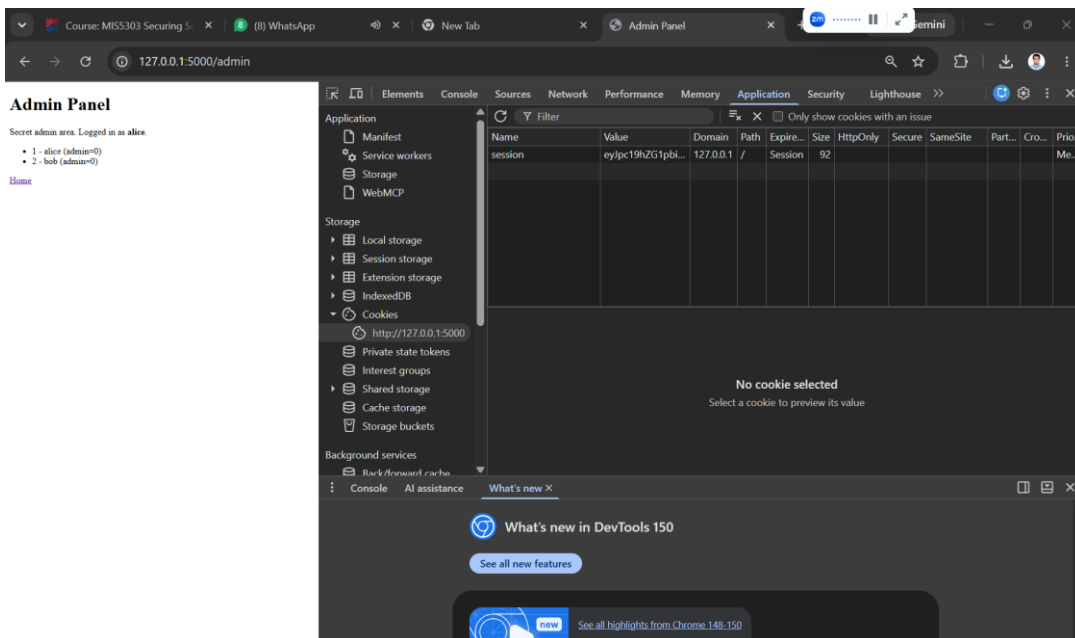


Figure 6b: `http://127.0.0.1:5000/make_admin` is now POST-only, so the GET URL attack returns HTTP 405 Method Not Allowed.

3. Architect's Plan — Secure Auth & Session Design

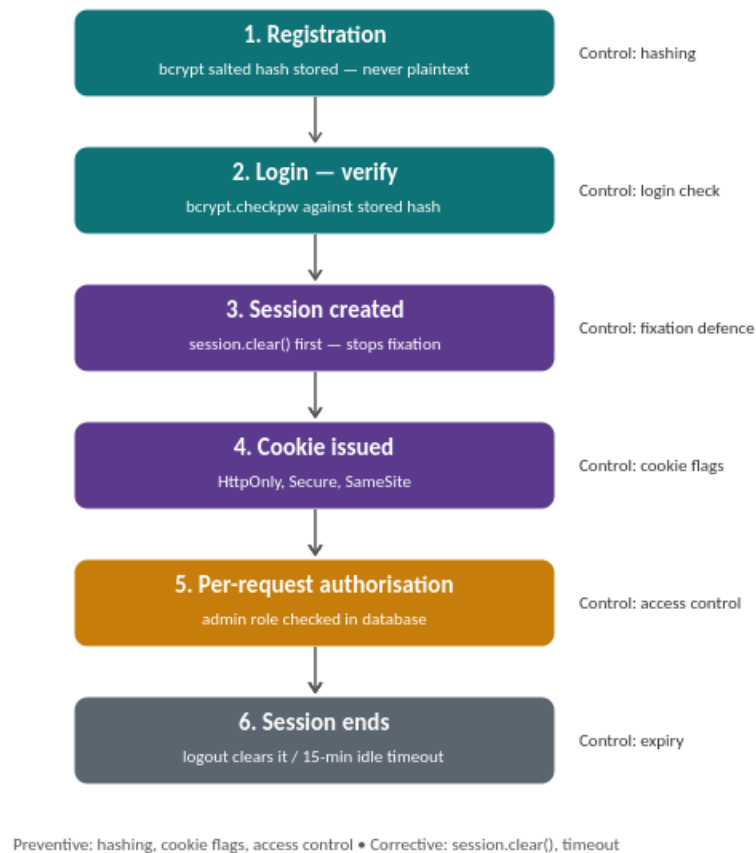


Figure 7: Secure authentication and session workflow, with the control applied at each stage.

Design decisions & justification

Question	Decision	Framework
Password policy	Min 8 (aim 12+), allow passphrases, screen against breached-password lists, no forced periodic rotation	NIST SP 800-63B
MFA / reset	Add TOTP MFA (e.g. pyotp); email-token password reset	Essential Eight (MFA)
Session expiry	15-min idle timeout + server-side invalidation on logout	OWASP Session Mgmt
Cookies / libs	HttpOnly+Secure+SameSite; Flask-Login for sessions, Flask-WTF for CSRF	OWASP / secure config

This application is protected at three layers, each guarding a different point. Password hashing protects the passwords stored in the database, the cookie flags protect the session as it travels between browser and server, and the admin permission check protects every request a user makes. The core idea is that no single fix covers every risk, so a control is needed at each layer - this is defence in depth.

On top of that I would add two-factor authentication, an email-based password reset, and sessions that expire after 15 minutes of inactivity. These align with NIST 800-63B password guidance and the Essential Eight, which lists multi-factor authentication as a core control.

4. Individual Reflection

In this lab I found and fixed three authentication and session problems. First, passwords were saved in the database exactly as typed - I could open the db and read `alicepw123` and `bobpw456` in plain text. If someone steals the database they get everyone's password. I fixed this with `bcrypt`, which scrambles the password into a hash that can't be reversed, so after the fix the database shows `$2b$12$telISOLVS...` instead of the real password.

Second, the session cookie had no protection flags. Without `HttpOnly` a malicious page script can steal the cookie, and without `SameSite` other websites can misuse it. After the fix the cookie shows `HttpOnly` enabled and `SameSite` set to `Lax`.

The hardest one to understand was the privilege escalation. The normal user `alice` could open the admin panel just by typing its URL, and could visit `http://127.0.0.1:5000/make_admin?user_id=1` to make herself admin - with no admin rights needed. It was hard because nothing in the code looked broken; the routes worked fine, they just never checked whether the user was allowed. I fixed it by adding a database check that returns 403 if the user isn't an admin, and by making `http://127.0.0.1:5000/make_admin` POST-only so a link can't trigger it.

I also learned that a fix isn't proven until you re-test it: I re-checked the database for the hash, tried the admin panel as a normal user and got 403, and tried the escalation URL and got 405. One limit of the tools is that a scanner like `Bandit` finds bad code patterns but can't detect a missing permission check - only reading the logic by hand catches that. These problems still happen in real systems because developers focus on making features work and forget to check who is allowed to use them.

6. Peer Review

Reviewer: Z N Zakaria (Student ID 240134). Peer feedback on this Lab 3 submission, provided during Week 4 session.

Suggestions

This security analysis delivers strong performance and perfectly aligns with two authoritative frameworks: NIST SP 800-63B and the Essential Eight. The prior adjustment to change the /makeadmin route's request method from GET to POST has effectively blocked simple URL-triggered attacks. If a CSRF token implemented via Flask-WTF is added, it can further prevent the risk of permission changes caused by cross-site form submissions.

Testing

The verification evidence you submitted before and after the fix is sufficiently persuasive, and it accurately covers all three identified categories of vulnerabilities: the adopted 12-bit bcrypt hash value, the HTTP 403 Forbidden status returned when accessing the `http://127.0.0.1:5000/admin` path, and the HTTP 405 Method Not Allowed status returned when a GET request is sent to /makeadmin. To further strengthen the rigor of the testing, you may add one additional test to verify that authenticated administrator users can successfully send POST requests to call the /makeadmin interface. This will prove that the core functions of the legitimate management process are not affected by the fix operations.

Polishing

The overall framework of the manuscript is clear, the logic of the flowcharts is smooth, and the labels of all illustrations are clear and standardized. The comparison table under subsection 2 in particular allows readers to quickly assess the effects of all fix measures briefly. Only one minor adjustment is needed: please unify the figure numbers in the main text to follow a continuous sequence. The current text contains both Figure 6 and 6b; adjusting them to the standard format of Figure 6, 7, 8 will keep the numbering neat and orderly.

Reviewer signature: Z N Zakaria Date: 01-Aug-2026

My response to the peer review

I agree with the previously proposed optimization suggestion of explicitly adding a CSRF token to the POST route of `http://127.0.0.1:5000http://127.0.0.1:5000/make_admin` to prevent cross-site form attacks. In the future, when I submit negative security test cases, I will also supplement verification evidence proving that legitimate administrator functions can still operate normally.